

Aprendizaje Profundo – Redes Neuronales Recurrentes (RNN – LSTM – GRU)

Técnicas de Inteligencia Artificial

Prof. Flavio Prieto
email: faprieto@unal.edu.co

INGENIERÍA MECATRONICA
Facultad de Ingeniería
Universidad Nacional de Colombia Sede Bogotá



5 de mayo de 2026

Recurrent Neural Networks (RNN)^{a b}.

Son un tipo de red neuronal diseñada para procesar **datos secuenciales**, donde el orden de las observaciones es importante, como en series de tiempo, texto o señales de voz.

^aBasado en el curso: [Deep Learning for Computer Vision](#).

^bBasado en: [Dive into Deep Learning](#).

Su arquitectura se basa en:

- ▶ **Celdas recurrentes:** permiten que la red mantenga y actualice un *estado interno* o memoria a lo largo de la secuencia.
- ▶ **Funciones de activación:** introducen no linealidad para modelar relaciones complejas.
- ▶ **Capas de salida:** generan predicciones en cada paso o al final de la secuencia.

Las RNN son útiles para tareas como **modelado de lenguaje**, **predicción de series temporales** y **reconocimiento de voz**.

Introducción.

Las Redes Neuronales Recurrentes (RNN) ofrecen gran flexibilidad para estructurar la arquitectura de la red neuronal.

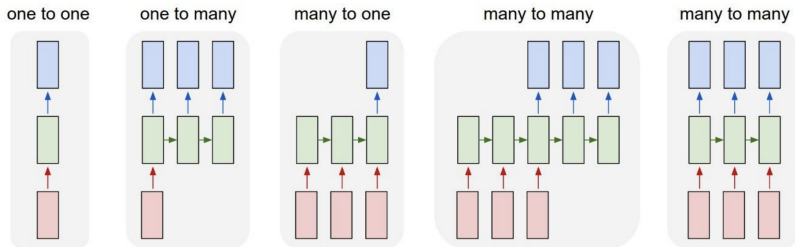
- ▶ En las **redes neuronales tradicionales**, la entrada tiene tamaño fijo y se procesa **mediante capas ocultas** para producir una salida de tamaño fijo.
- ▶ Cada entrada produce un único resultado o etiqueta.
- ▶ Es decir, se tiene un **mapeo uno a uno (one-to-one)**.

Las RNN permiten trabajar con secuencias de entradas, salidas, o ambos al mismo tiempo, ofreciendo distintos tipos de mapeos:

- ▶ Uno a muchos (one-to-many)
- ▶ Muchos a uno (many-to-one)
- ▶ Muchos a muchos (many-to-many)

Arquitectura de las redes neuronales.

- ▶ Imagen a la izquierda: Red neuronal tradicional.
- ▶ Cuatro imágenes a la derecha: Tipos de RNN.



- ▶ Las cajas rojas son **vectores de entrada**.
- ▶ Las cajas verdes son capas ocultas.
- ▶ Las cajas azules son vectores de salida.

Ejemplo de modelo One-to-Many:

- ▶ Un ejemplo típico es la **generación de descripciones de imágenes** (image captioning).
- ▶ Se da una imagen de tamaño fijo y se produce una secuencia de palabras que describen su contenido.
- ▶ Este modelo ilustra cómo una entrada única puede generar múltiples salidas a lo largo del tiempo.

Ejemplo de modelo Many-to-One:

- ▶ Un ejemplo es la **predicción de acciones** en videos:
 - ▶ Se observa una secuencia de fotogramas de video.
 - ▶ Se produce una etiqueta que describe la acción que ocurre en la secuencia.
- ▶ Otro ejemplo es la **clasificación de sentimientos** en NLP:
 - ▶ La entrada es una secuencia de palabras de una oración.
 - ▶ La salida es una clasificación de sentimiento (positiva o negativa).

Ejemplo de modelo Many-to-Many:

▶ Video-captioning:

- ▶ Entrada: secuencia de fotogramas de video.
- ▶ Salida: secuencia de palabras que describen el contenido del video.

▶ Traducción automática:

- ▶ Entrada: secuencia de palabras en inglés.
- ▶ Salida: secuencia de palabras en francés.

Variación de Many-to-Many: Predicción en cada timestep:

- ▶ La salida se genera en cada paso temporal.
- ▶ Ejemplo: clasificación de videos a nivel de fotograma.
- ▶ Cada predicción depende de:
 - ▶ El fotograma actual.
 - ▶ Todos los fotogramas anteriores de la secuencia.
- ▶ Esto ilustra la memoria y la capacidad de las RNN para modelar dependencias temporales.

Resumen

- ▶ Las RNN permiten que la predicción en cada paso temporal dependa de todos los pasos anteriores.
- ▶ Son ideales para datos secuenciales: texto, audio, video, series temporales.
- ▶ Diferentes configuraciones de mapeo permiten modelar tareas:
 - ▶ One-to-many
 - ▶ Many-to-one
 - ▶ Many-to-many

Comparación conceptual: MLP-CNNs vs RNNs

MLP o CNNs	RNNs
Entrada: tamaño fijo ($W \times H$) [Imagen 28x28]	Entrada: secuencia de longitud variable [Fotograma 1, Fotograma 2, ..., Fotograma T]
Capas: convolucionales [Conv $\rightarrow$ ReLU $\rightarrow$ Pool]	Capas: recurrentes Procesan un elemento de la secuencia a la vez y mantienen un estado oculto
Salida: tamaño fijo [Etiqueta única]	Salida: puede ser de longitud variable [Palabra 1, Palabra 2, ..., Palabra N]
Limitación: solo entradas de tamaño fijo	Ventaja: puede procesar secuencias de cualquier longitud

Aplicaciones de RNN

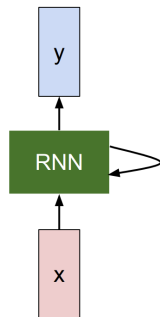
- ▶ Procesamiento de lenguaje natural: traducción automática, generación de texto.
- ▶ Series temporales: predicción de precios, análisis financiero.
- ▶ Audio y señales: reconocimiento de voz, síntesis de música.
- ▶ Video: clasificación de secuencias, predicción de frames futuros.

Redes Neuronales Recurrentes.

Una RNN puede considerarse como una “caja negra” que mantiene un **estado interno** actualizado a medida que se procesa una secuencia.

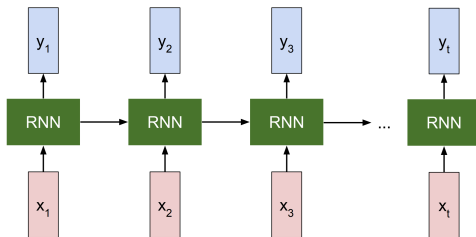
Funcionamiento básico de la RNN:

- ▶ En cada paso temporal, **se ingresa un vector de entrada** x_t a la RNN.
- ▶ El estado interno **se actualiza en función de la entrada y del estado previo**.
- ▶ La RNN produce un **vector de salida** y_t **basado en su estado actual**.
- ▶ Al ajustar los pesos de la RNN, se modifica cómo evoluciona el estado frente a diferentes secuencias de entrada.



RNN desplegada en el tiempo.

- ▶ La RNN se puede “desplegar” a lo largo de los pasos temporales.
- ▶ Entradas: $x_1, x_2, x_3, \dots, x_t$ (por ejemplo, fotogramas de un video).
- ▶ En cada paso, la RNN recibe:
 - ▶ La entrada actual x_i .
 - ▶ La representación previa (historia) de los pasos anteriores.
- ▶ Produce la salida y_i y actualiza su estado interno, que se propaga al siguiente paso.



Compartición de parámetros.

- ▶ Todos los bloques de la RNN son idénticos y comparten los mismos pesos.
- ▶ Cada bloque recibe diferentes entradas y estados previos en cada instante del tiempo.
- ▶ Esto permite que la RNN generalice a secuencias de longitud variable, manteniendo consistencia en la representación.

Fórmula de recurrencia de la RNN.

$$h_t = f_W(h_{t-1}, x_t)$$

- ▶ h_{t-1} : estado previo en el instante de tiempo $t - 1$.
- ▶ x_t : vector de entrada actual en el instante de tiempo t .
- ▶ h_t : estado actual producido por la RNN.
- ▶ f_W : función fija con pesos W , aplicada en cada instante de tiempo.

Propiedades de la función recurrente:

- ▶ La misma función f_W se aplica en todos los instantes de tiempo.
- ▶ Permite procesar secuencias de longitud variable sin cambiar la arquitectura.
- ▶ La salida o el estado se actualiza de manera consistente para cualquier longitud de secuencia.

La forma más simple de RNN se basa en un único estado oculto h que se actualiza en cada paso temporal, según la entrada actual y el estado previo.

Actualización del estado oculto.

En una RNN básica:

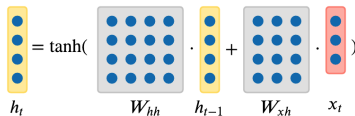
- ▶ Existen matrices de pesos: W_{hh} y W_{xh} .
- ▶ W_{hh} proyecta el estado oculto previo h_{t-1} .
- ▶ W_{xh} proyecta la entrada actual x_t .
- ▶ La suma de estas proyecciones se pasa por una función de activación, por ejemplo la $\tanh$, para actualizar el estado:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Interpretación de la recurrencia.

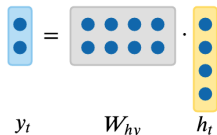
- ▶ La recurrencia indica cómo h_t cambia en función de:
 - ▶ La historia de estados previos h_{t-1} .
 - ▶ La entrada actual x_t .
- ▶ Permite que la red mantenga memoria de secuencias y aprenda dependencias temporales.

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



- ▶ Las predicciones se basan en h_t , utilizando otra proyección mediante una matriz posterior del estado oculto.

$$y_t = W_{hy}h_t$$



- ▶ Este es el caso más simple y completo de una RNN.

Formalismo matemático.

Redes neuronales sin estados ocultos.

- ▶ Considere un perceptrón multicapa (MLP) con una sola capa oculta, cuya función de activación es $\phi(\cdot)$.
- ▶ Dado un minibatch de muestras (entradas) $X \in \mathbb{R}^{n \times d}$, con
 - ▶ n , el tamaño de lote y
 - ▶ d , la dimensión de las entradas,la salida de la capa oculta $H \in \mathbb{R}^{n \times h}$, se calcula como:

$$H = \phi(XW_{xh} + b_h)$$

donde

- ▶ h indica el número de neuronas ocultas.
- ▶ $W_{xh} \in \mathbb{R}^{d \times h}$ representa los pesos de la capa, y
- ▶ $b_h \in \mathbb{R}^{1 \times h}$ es el vector de sesgos.

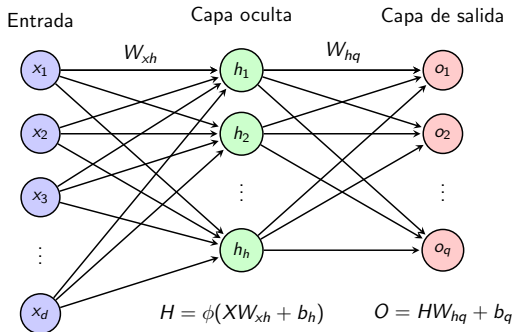
- ▶ La salida de la capa oculta H se utiliza como entrada de la capa de salida, definida como:

$$O = HW_{hq} + b_q$$

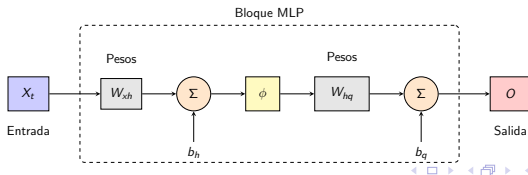
donde:

- ▶ $O \in \mathbb{R}^{n \times q}$ corresponde a la variable de salida,
 - ▶ $W_{hq} \in \mathbb{R}^{h \times q}$ es la matriz de pesos de la capa de salida y
 - ▶ $b_q \in \mathbb{R}^{1 \times q}$ es el sesgo correspondiente.
-
- ▶ Si se trata de un problema de clasificación, se aplica la función **softmax** a O para obtener la distribución de probabilidad sobre las categorías de salida.

MLP con una capa oculta.



$$X \in \mathbb{R}^{n \times d}$$



Redes neuronales con estados ocultos.

- ▶ El escenario cambia cuando se introducen **estados ocultos**.
- ▶ Se analiza la estructura con mayor detalle para comprender su funcionamiento.
- ▶ Supóngase un **minibatch de entradas** en el paso de tiempo t :

$$X_t \in \mathbb{R}^{n \times d}$$

donde:

- ▶ n corresponde al tamaño del lote (número de secuencias).
- ▶ d corresponde a la dimensión de cada entrada en la secuencia.

Estados ocultos en RNN.

- ▶ Denótese por $H_t \in \mathbb{R}^{n \times h}$ la **salida de la capa oculta** en el paso de tiempo t .
- ▶ A diferencia del MLP, **en RNN se guarda H_{t-1} y se introduce un nuevo parámetro de pesos $W_{hh} \in \mathbb{R}^{h \times h}$ para usar la salida oculta anterior.**
- ▶ La salida oculta actual se calcula como:

$$H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$

- ▶ Comparado con el MLP, se añade el término $\boxed{H_{t-1} W_{hh}}$.

Interpretación de los estados ocultos.

- ▶ La relación entre H_t y H_{t-1} implica que los estados ocultos:
 - ▶ Capturan y retienen la **información histórica de la secuencia**.
 - ▶ Funcionan como **estado o memoria** de la red neuronal.
- ▶ Dado que H_{t-1} influye en H_t , el cálculo es **recurrente**.
- ▶ Por lo tanto, redes con estados ocultos definidos mediante este mecanismo se denominan **Redes Neuronales Recurrentes (RNN)**.
- ▶ Las capas de esta red se conocen como **capas recurrentes**.

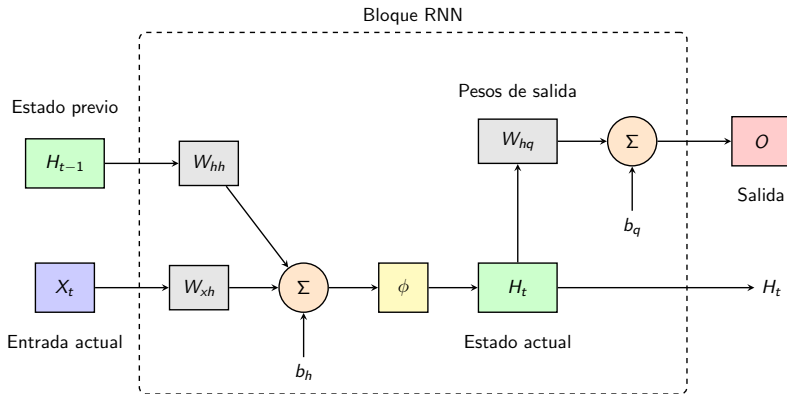
Cálculo de la salida en RNN.

- ▶ Para un paso de tiempo t , la salida de la capa de salida se obtiene de forma similar a un MLP:

$$O_t = H_t W_{hq} + b_q$$

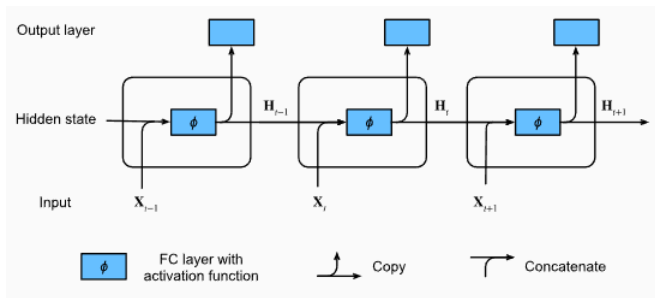
- ▶ Parámetros del modelo:
 - ▶ Pesos $W_{xh} \in \mathbb{R}^{d \times h}$ y sesgo $b_h \in \mathbb{R}^{1 \times h}$ de la capa oculta.
 - ▶ Pesos $W_{hq} \in \mathbb{R}^{h \times q}$ y sesgo $b_q \in \mathbb{R}^{1 \times q}$ de la capa de salida.
 - ▶ Matriz recurrente $W_{hh} \in \mathbb{R}^{h \times h}$.

Bloque recurrente.



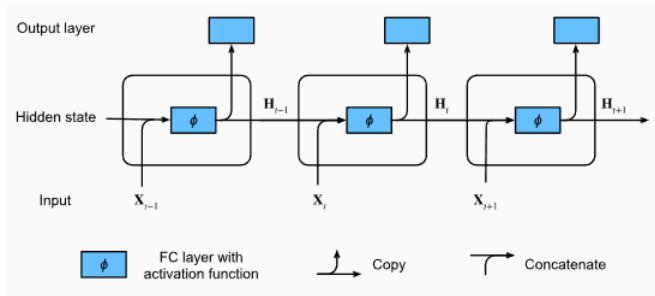
Lógica computacional de una RNN en tres pasos de tiempo.

- ▶ En cualquier paso t , el cálculo del estado oculto puede interpretarse como:
 1. Concatenar la entrada X_t en el paso de tiempo actual con el estado oculto H_{t-1} del paso anterior.
 2. Alimentar el resultado de dicha concatenación en una capa totalmente conectada con función de activación ϕ .
- ▶ La salida de esta capa totalmente conectada corresponde al estado oculto H_t en el paso actual.



Redes Neuronales Recurrentes

- ▶ En este caso, los parámetros del modelo corresponden a la concatenación de W_{xh} y W_{hh} , junto con un sesgo b_h .
- ▶ El estado oculto H_t no solo participa en el cálculo del estado oculto siguiente H_{t+1} , sino que también se introduce en la capa de salida totalmente conectada para calcular la salida O_t .



Entrenamiento de las RNN.

Definiciones:

En una RNN, para cada instante de tiempo t se definen:

$$H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$

$$O_t = f(H_t W_{hq} + b_q)$$

- ▶ X_t : entrada en el tiempo t .
- ▶ H_t : estado oculto en el tiempo t .
- ▶ O_t : salida en el tiempo t .
- ▶ ϕ : función de activación recurrente.
- ▶ f : función de activación de salida.

Parámetros del modelo:

$$\theta = \{W_{xh}, W_{hh}, W_{hq}, b_h, b_q\}$$

Función de Pérdida.

La pérdida total sobre toda la secuencia se define como:

$$\mathcal{L} = \sum_{t=1}^T \ell(Y_t, O_t)$$

- ▶ Y_t : etiqueta esperada en el instante t .
- ▶ ℓ : función de pérdida elemental (p.ej. entropía cruzada).
- ▶ T : longitud de la secuencia.

Propagación hacia Adelante.

Durante la propagación hacia adelante:

1. Se calcula el estado oculto H_t para cada paso de tiempo.
2. Se obtiene la salida O_t .
3. Se acumula la contribución a la pérdida $\ell(Y_t, O_t)$.

Retropropagación a través del Tiempo (BPTT).

El error se propaga hacia atrás no solo en las capas, sino también en el tiempo:

$$\delta_t = \left(\frac{\partial \ell_t}{\partial O_t} W_{hq}^\top + \delta_{t+1} W_{hh}^\top \right) \odot \phi'(Z_t)$$

$$Z_t = X_t W_{xh} + H_{t-1} W_{hh} + b_h$$

- ▶ δ_t : error en el estado oculto en el tiempo t .
- ▶ $\odot$: producto elemento a elemento.

Gradientes de los Parámetros.

Los gradientes se obtienen acumulando contribuciones a lo largo del tiempo:

$$\frac{\partial \mathcal{L}}{\partial W_{hq}} = \sum_{t=1}^T H_t^\top \frac{\partial \ell_t}{\partial O_t}$$

$$\frac{\partial \mathcal{L}}{\partial W_{xh}} = \sum_{t=1}^T X_t^\top \delta_t,$$

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^T H_{t-1}^\top \delta_t$$

$$\frac{\partial \mathcal{L}}{\partial b_h} = \sum_{t=1}^T \delta_t,$$

$$\frac{\partial \mathcal{L}}{\partial b_q} = \sum_{t=1}^T \frac{\partial \ell_t}{\partial O_t}$$

El algoritmo de entrenamiento en una RNN sigue estos pasos:

1. Inicializar parámetros θ y estado oculto H_0 .
2. Para cada secuencia:
 - ▶ **Forward pass:** calcular H_t, O_t .
 - ▶ **Compute loss:** $\mathcal{L} = \sum_{t=1}^T \ell(Y_t, O_t)$.
 - ▶ **Backward pass (BPTT):** calcular gradientes.
 - ▶ **Update:** ajustar parámetros.

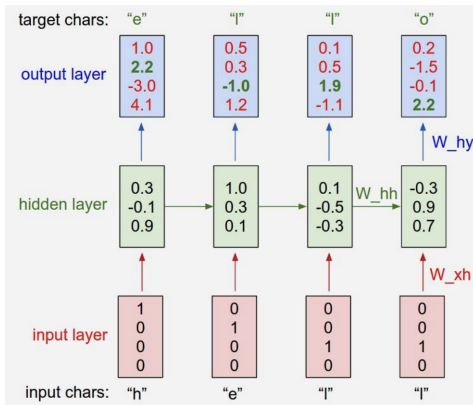
Ejemplo: modelo de lenguaje a nivel de caracteres.

- ▶ La RNN recibe una secuencia de caracteres.
- ▶ En cada instante del tiempo la red debe predecir el siguiente carácter en la secuencia.
- ▶ La salida es un vector de puntuaciones (logits) para cada carácter del vocabulario, indicando qué carácter cree la RNN que debería aparecer a continuación en la secuencia que ha visto hasta ese momento.

Redes Neuronales Recurrentes

Ejemplo de secuencia "hello"

- ▶ Secuencia de entrenamiento: "hello"
- ▶ Vocabulario: $V = \{h, e, l, o\}$, de 4 caracteres.
- ▶ Cada carácter se codifica con un **one-hot vector**:



Codificación de Caracteres.

- ▶ Como se muestra en la figura, se alimenta un carácter a la vez a la RNN: primero “h”, luego “e”, después “l” y finalmente “l”.
- ▶ Todos los caracteres se codifican mediante un **vector one-hot**, donde solo un bit del vector está activado para cada carácter del vocabulario.

Por ejemplo:

$$h = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad e = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad l = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad o = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Aplicando la recurrencia:

- ▶ Estado inicial: $h_0 = [0, 0, 0]^T$.
- ▶ Para cada instante de tiempo t , se actualiza el estado oculto h_t usando la fórmula de recurrencia:

$$h_t = f_W(W_{hh}h_{t-1} + W_{xh}x_t)$$

- ▶ Ejemplo de actualizaciones para la secuencia “hello”:

$$h_1 = f_W(W_{hh}[0, 0, 0]^T + W_{xh}[1, 0, 0, 0]^T) = [0.3, -0.1, 0.9]^T$$

$$h_2 = f_W(W_{hh}h_1 + W_{xh}[0, 1, 0, 0]^T) = [1.0, 0.3, 0.1]^T$$

$$h_3 = f_W(W_{hh}h_2 + W_{xh}[0, 0, 1, 0]^T) = [0.1, -0.5, -0.3]^T$$

$$h_4 = f_W(W_{hh}h_3 + W_{xh}[0, 0, 1, 0]^T) = [-0.3, 0.9, 0.9]^T$$

Predicción de caracteres.

- ▶ En cada instante de tiempo, la RNN predice el siguiente carácter en forma de vector de logits:

$$\begin{bmatrix} 1.0 \\ 2.2 \\ -3.0 \\ 4.1 \end{bmatrix} \longrightarrow \begin{bmatrix} \text{"h"} \\ \text{"e"} \\ \text{"l"} \\ \text{"o"} \end{bmatrix}$$

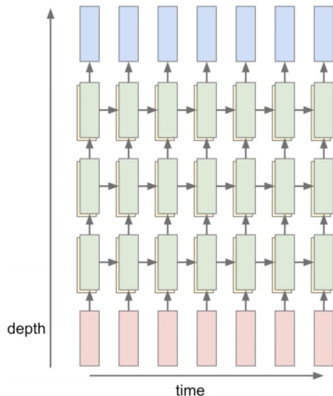
- ▶ El vector indica la probabilidad de cada carácter según la RNN: $\text{prob}(\text{h})=1.0$, $\text{prob}(\text{e})=2.2$, $\text{prob}(\text{i})=-3.0$, y $\text{prob}(\text{o})=4.1$.
- ▶ En el ejemplo, la RNN predice incorrectamente "o" como más probable, pero la correcta es "l".

Multilayer RNNs

- ▶ Hasta ahora se han mostrado RNNs con una sola capa.
- ▶ Sin embargo, las RNNs no se limitan a arquitecturas de una capa.
 - ▶ Hoy en día, es común usar RNNs de manera más compleja, apilando varias capas.
 - ▶ Las RNNs multilayer permiten mayor profundidad, y empíricamente, arquitecturas más profundas tienden a funcionar mejor.

Ejemplo de RNN multicapa.

- ▶ En la figura hay tres RNNs separadas, cada una con su propio conjunto de pesos.
- ▶ Las RNNs se apilan, de modo que la entrada de la segunda capa RNN es el vector de estado oculto de la primera capa.
- ▶ Todas las capas apiladas se entrenan conjuntamente.
- ▶ El diagrama representa un único grafo computacional para la RNN multilayer.



Ejemplo Python

- ▶ Ver código: **Ejemplo RNN.**

Capas Recurrentes en Keras / TensorFlow:

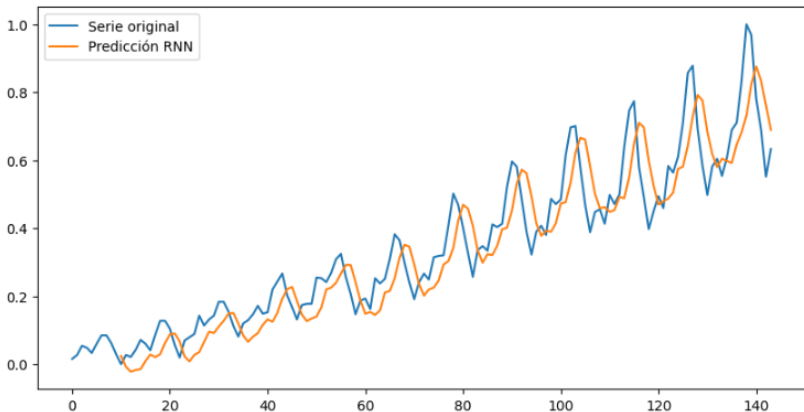
- ▶ SimpleRNN: RNN estándar, cada neurona recibe la salida anterior y la entrada actual.

Capas Recurrentes en PyTorch:

- ▶ `nn.RNN`.
- ▶ Soportan secuencias de longitud variable y multi-capas (`num_layers`).
- ▶ Salida: todos los estados ocultos o solo el último (`return_sequence, return_state`).

Ejemplo:

- ▶ RNN para predecir la evolución de una serie temporal real: el dataset AirPassengers, que contiene el número mensual de pasajeros de aerolíneas entre 1949 y 1960.



Problemas de las RNN estándar

- ▶ **Desvanecimiento del gradiente:** los gradientes tienden a cero en secuencias largas.
- ▶ **Explosión del gradiente:** los gradientes pueden crecer demasiado, causando inestabilidad.
- ▶ Soluciones:
 - ▶ Uso de LSTM o GRU.
 - ▶ Clipping de gradientes.
 - ▶ Inicialización adecuada de pesos.

Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) ^a ^b.

Son una variante de las redes recurrentes (RNN) diseñada para **capturar dependencias a largo plazo** y mitigar el problema del *desvanecimiento del gradiente*.

^aBasado en el curso: [Deep Learning for Computer Vision](#).

^bBasado en: [Dive into Deep Learning](#).

Long Short-Term Memory (LSTM)

El término **memoria a largo y corto plazo** proviene de la siguiente intuición:

- ▶ Las redes neuronales recurrentes simples tienen **memoria a largo plazo en forma de pesos**.
- ▶ **Los pesos cambian lentamente** durante el entrenamiento, codificando conocimiento general sobre los datos.
- ▶ También tienen **memoria a corto plazo en forma de activaciones efímeras**, que se transmiten de cada nodo a los nodos sucesivos.
- ▶ **El modelo LSTM introduce un tipo intermedio de almacenamiento mediante la célula de memoria**.
- ▶ Una célula de memoria es una unidad compuesta, construida a partir de nodos más simples en un patrón de conectividad específico, con la **inclusión novedosa de nodos multiplicativos**.

Long Short-Term Memory (LSTM)

Su arquitectura incorpora:

- ▶ **Celdas de memoria:** almacenan información relevante a lo largo de la secuencia.
- ▶ **Puertas de entrada, olvido y salida:** regulan el flujo de información que entra, se mantiene o sale de la celda.
- ▶ **Estado oculto:** representa la memoria a corto plazo utilizada para la predicción.

Las LSTM han tenido gran éxito en aplicaciones como **traducción automática, generación de texto, análisis de sentimientos y predicción de series temporales.**

Problema de desvanecimiento del gradiente.

- ▶ Un bloque RNN recibe como entrada x_t y la representación oculta previa h_{t-1} .
- ▶ Aprende una transformación que se pasa a través de $\tanh$ para producir:
 - ▶ La representación oculta h_t para el siguiente paso de tiempo.
 - ▶ La salida y_t .
- ▶ Ecuación de actualización de la RNN estándar:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Long Short-Term Memory (LSTM)

- ▶ Para la **retropropagación**, se analiza cómo la salida del último paso de tiempo afecta los pesos del primer paso:

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

- ▶ Actualización de los pesos W_{hh} usando la derivada de la pérdida L_t :

$$\frac{\partial L_t}{\partial W_{hh}} = \frac{\partial L_t}{\partial h_t} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W_{hh}}$$

- ▶ $\tanh'(W_{hh}h_{t-1} + W_{xh}x_t)$ es usualmente menor que 1.
- ▶ A medida que t crece, el gradiente $\frac{\partial L_t}{\partial W_{hh}}$ **tiende a cero**.
- ▶ **Consecuencia:** los gradientes de pasos futuros raramente afectan los primeros pasos.
- ▶ Esto ralentiza enormemente el aprendizaje en secuencias largas.

Problema:

- ▶ Eliminando la no linealidad tanh, el gradiente sería:

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial h_t} \left(\prod_{t=2}^T W_{hh}^{T-1} \right) \frac{\partial h_1}{\partial W}$$

- ▶ **Gradientes explosivos:**

- ▶ Ocurren si el **valor singular más grande de W_{hh} es mayor que 1.**
- ▶ Provocan gradientes extremadamente grandes o NaNs.

- ▶ **Gradientes desvanecientes:**

- ▶ Ocurren si el **valor singular más grande de W_{hh} es menor que 1.**
- ▶ Ralentizan significativamente el aprendizaje.

Solución: LSTM.

- ▶ Se puede tratar el problema de gradientes explosivos mediante “gradient clipping”.
- ▶ Sin embargo, el problema de gradientes desvanecientes persiste en secuencias largas.
- ▶ **LSTM está diseñado para evitar el problema de gradientes desvanecientes.**
- ▶ La arquitectura de LSTM permite que los gradientes fluyan a través del tiempo sin desaparecer.

Estado oculto con compuertas.

- ▶ La distinción clave entre las RNN tradicionales y las LSTM es que estas últimas permiten **la compuerta del estado oculto**.
- ▶ Esto significa que existen **mecanismos dedicados tanto para decidir cuándo se debe actualizar un estado oculto como para decidir cuándo debe reiniciarse**.
- ▶ Estos mecanismos se **aprenden automáticamente** y abordan los problemas mencionados anteriormente.
- ▶ Por ejemplo:
 - ▶ Si el primer token es de gran importancia, se aprenderá a **no actualizar** el estado oculto después de la primera observación.
 - ▶ De manera similar, se aprenderá a **ignorar observaciones temporales irrelevantes**.
 - ▶ Por último, se aprenderá a **reiniciar el estado latente** siempre que sea necesario.

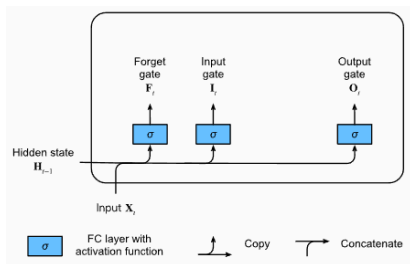
Formulación de LSTM.

- ▶ LSTM tiene una celda adicional respecto a una RNN estándar, C_t , que permite almacenar información a largo plazo.
- ▶ LSTM puede leer, borrar y escribir información en esta celda C_t .
 - ▶ LSTM modifica la celda C_t mediante tres compuertas especiales:
 - ▶ i – compuerta de **entrada** (*input gate*).
 - ▶ f – compuerta de **olvido** (*forget gate*).
 - ▶ o – compuerta de **salida** (*output gate*).
 - ▶ Los valores de las puertas varían de cerrado (0) a abierto (1).
 - ▶ Cada puerta es un vector de tamaño n .

Long Short-Term Memory (LSTM)

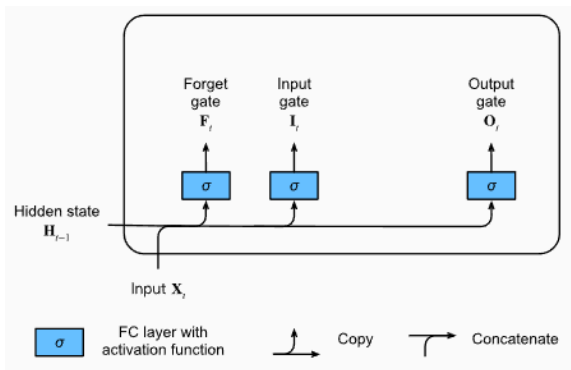
Cálculo de Estados en LSTM.

- ▶ Los datos que alimentan las compuertas de la LSTM son **la entrada en el paso de tiempo actual y el estado oculto del paso de tiempo anterior.**
- ▶ Tres capas totalmente conectadas con funciones de activación sigmoide calculan los valores de las compuertas de **entrada, olvido y salida.**
- ▶ Como resultado de la activación sigmoide, todos los valores de estas tres compuertas se encuentran en el rango $[0, 1]$.



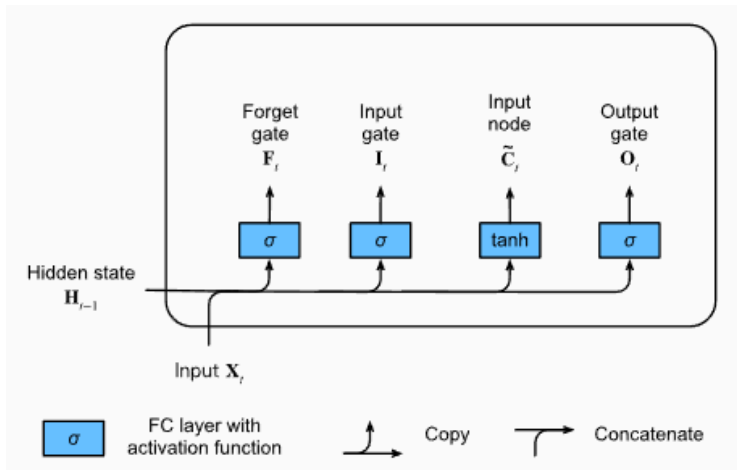
Long Short-Term Memory (LSTM)

- ▶ La **compuerta de entrada** determina qué proporción del valor del nodo de entrada debe sumarse al estado interno actual de la celda de memoria.
- ▶ La **compuerta de olvido** decide si se mantiene el valor actual de la memoria o se elimina.
- ▶ La **compuerta de salida** determina si la celda de memoria influirá en la salida del paso de tiempo actual.



Long Short-Term Memory (LSTM)

- Adicionalmente, se requiere un **nodo de entrada**, típicamente calculado con una función de activación *tanh*.



Formulación matemática.

- ▶ Suponga una red con:
 - ▶ h unidades ocultas,
 - ▶ tamaño de lote de n , y
 - ▶ el número de entradas es d .
- ▶ Así, la entrada en el tiempo t es $X_t \in \mathbb{R}^{n \times d}$
- ▶ y el estado oculto del paso anterior es $H_{t-1} \in \mathbb{R}^{n \times h}$.

Correspondientemente, las puertas en el instante t se definen de la siguiente manera:

- ▶ Puerta de entrada: I_t
- ▶ Puerta de olvido: F_t
- ▶ Puerta de salida: O_t

Long Short-Term Memory (LSTM)

Estas se calculan de la siguiente forma:

$$\begin{aligned}I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i), \\F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f), \\O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o),\end{aligned}$$

donde:

- ▶ $W_{xi}, W_{xf}, W_{xo} \in \mathbb{R}^{d \times h}$ y $W_{hi}, W_{hf}, W_{ho} \in \mathbb{R}^{h \times h}$ representan parámetros de peso y
- ▶ $b_i, b_f, b_o \in \mathbb{R}^{1 \times h}$ corresponden a los parámetros de sesgo.
- ▶ Se utilizan funciones sigmoides $\sigma(\cdot)$ para mapear los valores de entrada al intervalo $(0, 1)$.

Diseño de la Celda de Memoria.

Para la celda de memoria, se introduce el *nodo de entrada* $\tilde{C}_t \in \mathbb{R}^{n \times h}$.

- ▶ Su cálculo es semejante al de las tres compuertas descritas anteriormente, pero en este caso se emplea la función $\tanh(\cdot)$ como función de activación, cuyo rango de valores pertenece al intervalo $(-1, 1)$.
- ▶ Esto conduce a la siguiente ecuación en el instante t :

$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c),$$

donde $W_{xc} \in \mathbb{R}^{d \times h}$ representa parámetros de peso y $b \in \mathbb{R}^{1 \times h}$ corresponde a un parámetro de sesgo.

Estado Interno de la Celda de Memoria.

En las LSTM:

- ▶ La compuerta de entrada I_t controla **cuánto se tiene en cuenta la nueva información** a través de $\tilde{C}_t$, mientras que
- ▶ la puerta de olvido F_t regula **cuánto del estado interno anterior C_{t-1} se conserva.**

Utilizando el producto de Hadamard¹, se obtiene la siguiente **ecuación de actualización**:

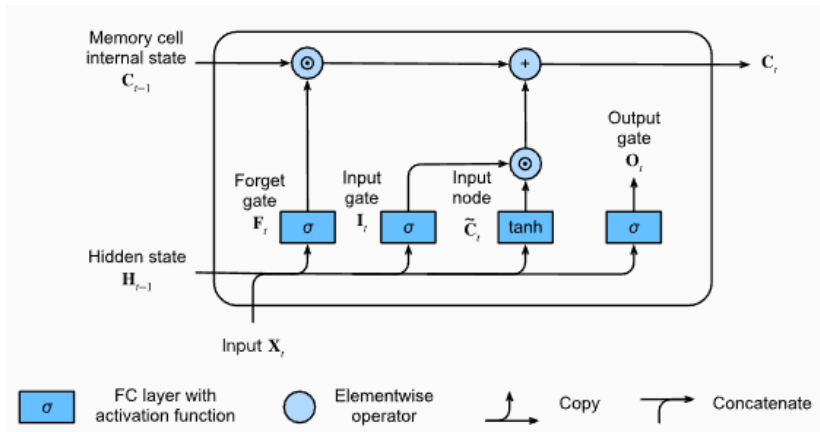
$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t,$$

- ▶ donde $\odot$ representa el producto elemento a elemento.

¹producto elemento a elemento.

Long Short-Term Memory (LSTM)

Estado Interno de la Celda de Memoria.



Long Short-Term Memory (LSTM)

- ▶ Si la puerta de olvido es siempre 1 y la puerta de entrada siempre 0, el estado interno C_t permanece constante indefinidamente, propagándose sin cambios en cada paso de tiempo.
- ▶ Sin embargo, al permitir que estas puertas aprendan dinámicamente cuándo mantener inalterado el valor y cuándo modificarlo en respuesta a nuevas entradas, el modelo obtiene la flexibilidad necesaria para representar dependencias de largo plazo.
- ▶ En la práctica, este diseño mitiga el **problema del gradiente desvanecido**, lo que facilita el entrenamiento de modelos, especialmente cuando se trabaja con secuencias largas.

Estado Oculto.

Cálculo de la salida de la celda de memoria, es decir, el estado oculto $H_t \in \mathbb{R}^{n \times h}$, el cual es visible para las demás capas.

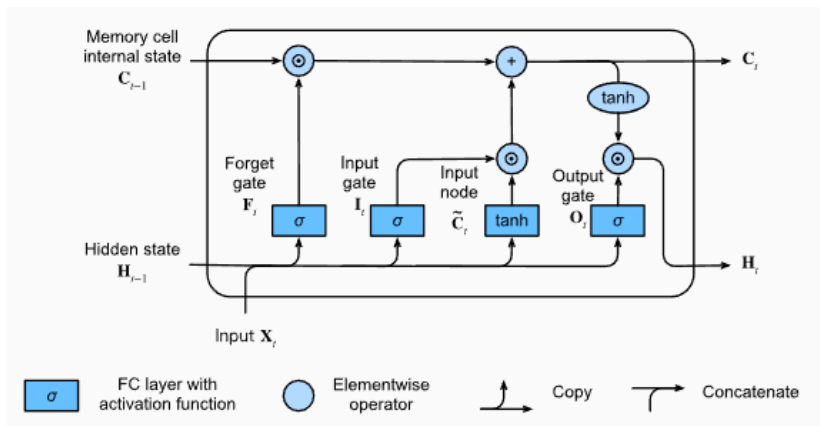
- ▶ En este punto entra en juego la *puerta de salida* O_t .
- ▶ En las LSTM,
 - ▶ primero se aplica la función $\tanh(\cdot)$ al estado interno de la celda C_t , y
 - ▶ posteriormente se realiza una multiplicación punto a punto con la puerta de salida.
 - ▶ Esto asegura que los valores de H_t se mantengan dentro del intervalo $(-1, 1)$.
- ▶ La ecuación de actualización es la siguiente:

$$H_t = O_t \odot \tanh(C_t),$$

- ▶ donde $\odot$ representa el producto elemento a elemento.

Long Short-Term Memory (LSTM)

Estado Oculto.



Long Short-Term Memory (LSTM)

- ▶ Cuando la puerta de salida O_t está cercana a 1,
 - ▶ el estado interno de la celda impacta directamente en las capas posteriores.
- ▶ En contraste, si O_t se aproxima a 0,
 - ▶ se evita que la memoria actual influya en otras capas de la red en ese instante de tiempo.

De esta manera,

- ▶ una celda de memoria puede acumular información a lo largo de múltiples pasos temporales sin impactar al resto de la red (mientras $O_t \approx 0$), y
- ▶ luego influir de manera significativa cuando la puerta de salida cambia repentinamente de valores cercanos a 0 hacia valores próximos a 1.

Resumen de interpretación:

- ▶ $F_t \odot C_{t-1}$ **mantiene** información previa según la compuerta de olvido.
- ▶ $I_t \odot \tilde{C}_t$ **agrega** nueva información filtrada por la compuerta de entrada.
- ▶ $\tanh(C_t)$ **normaliza** la información contenida en la celda.
- ▶ La puerta de salida O_t **decide** qué parte de C_t se revela al siguiente paso de tiempo.

Este mecanismo permite que la LSTM controle de manera precisa el flujo de información hacia la salida y hacia pasos futuros.

¿LSTM Resuelve el Problema de Gradientes Desvanecientes?

- ▶ La arquitectura de LSTM facilita que la RNN **preserve información** a lo largo de muchos pasos de tiempo recurrentes.
- ▶ Ejemplo:
 - ▶ Si la **puerta de olvido** f_t se establece en 1 y la **puerta de entrada** i_t en 0, la información del estado de celda c_t se preserva durante muchos pasos.
 - ▶ En contraste, en una RNN estándar es mucho más difícil mantener información en los estados ocultos usando solo una matriz de pesos.
- ▶ LSTMs **no garantizan la eliminación total de los problemas de gradientes desvanecientes o explosivos**, pero proporcionan una manera más eficiente para que el modelo aprenda dependencias de largo plazo.

Número de Parámetros en una LSTM

En una LSTM, cada celda contiene **cuatro componentes principales** que requieren parámetros:

- ▶ La compuerta de entrada I_t
- ▶ La compuerta de olvido F_t
- ▶ La compuerta de salida O_t
- ▶ El nodo de entrada (o candidato de memoria) $\tilde{C}_t$

Cada uno de estos componentes posee sus propios parámetros de pesos y sesgos.

Long Short-Term Memory (LSTM)

Parámetros Asociados a Cada Compuerta.

Para cada puerta (o nodo candidato) se deben calcular:

- ▶ Matriz de pesos de entrada: $W_x \in \mathbb{R}^{d \times h}$
- ▶ Matriz de pesos ocultos: $W_h \in \mathbb{R}^{h \times h}$
- ▶ Vector de sesgo: $b \in \mathbb{R}^h$

Por tanto, el número de parámetros para una compuerta es:

$$P_{\text{puerta}} = d \cdot h + h \cdot h + h$$

donde:

- ▶ d es la dimensión de entrada y
- ▶ h es el número de unidades ocultas.

Dado que la LSTM tiene **tres puertas** (I_t, F_t, O_t) y **un nodo candidato** ($\tilde{C}_t$), el **número total de parámetros** es:

$$P_{\text{LSTM}} = 4 \cdot (d \cdot h + h \cdot h + h)$$

Ejemplo Numérico: Red LSTM Completa.

Considérese una red LSTM con:

- ▶ Dimensión de entrada: $d = 50$
- ▶ Unidades ocultas: $h = 100$
- ▶ Capa de salida **densa** con $k = 10$ clases

$$\begin{aligned}P_{\text{LSTM}} &= 4 \cdot (d \cdot h + h \cdot h + h) = 4 \cdot (50 \cdot 100 + 100 \cdot 100 + 100) \\&= 4 \cdot (5000 + 10000 + 100) = 4 \cdot 15100 = 60400\end{aligned}$$

Además, la capa de salida requiere:

$$P_{\text{out}} = h \cdot k + k = 100 \cdot 10 + 10 = 1010$$

$$P_{\text{total}} = 60400 + 1010 = 61410$$

Ventajas y Limitaciones de LSTM.

Ventajas:

- ▶ Preservan información a largo plazo (estado de celda c_t).
- ▶ Mitigan significativamente el problema de gradientes desvanecientes.
- ▶ Control fino del flujo de información mediante puertas de entrada, olvido y salida.
- ▶ Eficaces en tareas de secuencias largas como procesamiento de lenguaje, series temporales y reconocimiento de voz.

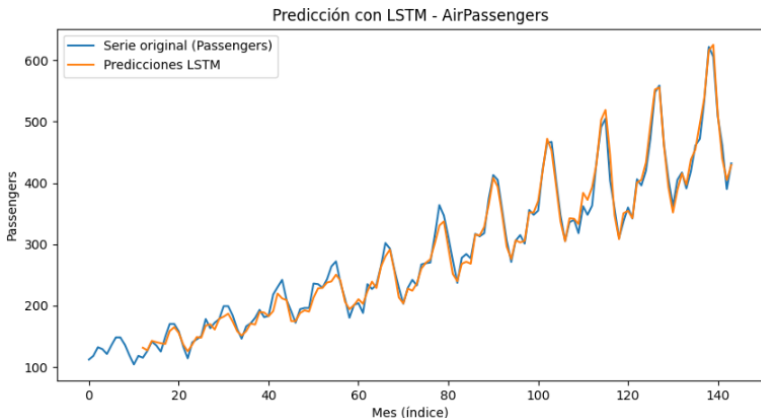
Limitaciones:

- ▶ Más complejas que las RNN estándar, con mayor cantidad de parámetros y cálculos.
- ▶ No garantizan la eliminación total de gradientes desvanecientes o explosivos.
- ▶ Entrenamiento más lento y mayor requerimiento de memoria.
- ▶ Más difíciles de interpretar que otros modelos más simples.

Long Short-Term Memory (LSTM)

Ejemplo Python

- ▶ Ver código: **Ejemplo LTSM.**
- ▶ Evolución de la serie temporal AirPassengers con LSTM.



Gated Recurrent Unit (GRU)^a.

Son una variante simplificada de las LSTM que también están diseñadas para **capturar dependencias a largo plazo** en secuencias, pero con una estructura más compacta.

^aBasado en: [Dive into Deep Learning](#).

Su arquitectura incluye:

- ▶ **Compuerta de actualización:** decide qué parte de la memoria previa se conserva.
- ▶ **Compuerta de reinicio:** controla qué parte de la memoria pasada se combina con la nueva entrada.
- ▶ **Estado oculto:** almacena la información relevante para cada paso de la secuencia.

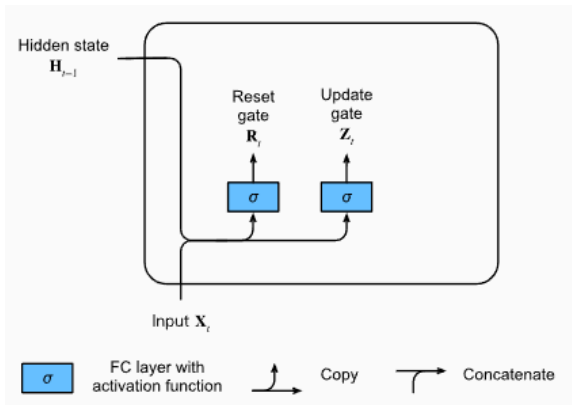
Las GRU son populares por su **simplicidad**, menor coste computacional y eficacia en tareas como **modelado de lenguaje**, **traducción automática** y **predicción de series temporales**.

Compuerta de reinicio y Compuerta de actualización.

- ▶ En una GRU, las tres puertas de la LSTM se reemplazan por dos: la Compuerta de reinicio (**reset gate**) y la Compuerta de actualización (**update gate**).
- ▶ Al igual que en las LSTM, estas compuertas utilizan activaciones sigmoideas, forzando sus valores a estar en el intervalo $[0, 1]$.
- ▶ Intuitivamente:
 - ▶ La **reset gate** controla cuánto del estado previo aún se debe recordar.
 - ▶ La **update gate** regula cuánto del nuevo estado es simplemente una copia del anterior.

Reset Gate y Update Gate en la GRU

- ▶ La figura ilustra las entradas de ambas compuertas, dadas por:
 - ▶ La entrada actual X_t .
 - ▶ El estado oculto previo H_{t-1} .
- ▶ Las salidas de ambas compuertas están dadas por capas totalmente conectadas con activación sigmoide.



Definición Matemática de las Compuertas.

En un paso de tiempo t , se considera:

- ▶ Una entrada en forma de minibatch $X_t \in \mathbb{R}^{n \times d}$
 - ▶ n : número de ejemplos (muestras),
 - ▶ d : número de características de entrada.
- ▶ Un estado oculto previo $H_{t-1} \in \mathbb{R}^{n \times h}$
 - ▶ h : número de unidades ocultas.

Entonces, las dos compuertas de la GRU se definen como:

- ▶ La **reset gate** $R_t \in \mathbb{R}^{n \times h}$.
- ▶ La **update gate** $Z_t \in \mathbb{R}^{n \times h}$.

$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r)$$

$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

- ▶ $W_{xr}, W_{xz} \in \mathbb{R}^{d \times h}$ y $W_{hr}, W_{hz} \in \mathbb{R}^{h \times h}$ son matrices de pesos.
- ▶ $b_r, b_z \in \mathbb{R}^{1 \times h}$ son vectores de sesgo.
- ▶ $\sigma(\cdot)$ función sigmoide que restringe los valores al intervalo $[0, 1]$.

Estado Oculto Candidato.

- ▶ Se integra la **reset gate** R_t con el mecanismo de actualización regular de una RNN.
- ▶ Esto conduce al cálculo del **estado oculto candidato** $\tilde{H}_t \in \mathbb{R}^{n \times h}$ en el paso de tiempo t .
- ▶ La función de activación empleada es $\tanh(\cdot)$.

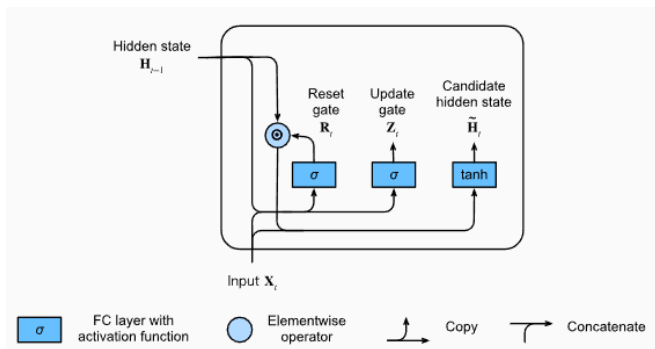
$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

donde:

- ▶ $W_{xh} \in \mathbb{R}^{d \times h}$ y $W_{hh} \in \mathbb{R}^{h \times h}$ son matrices de pesos y
- ▶ $b_h \in \mathbb{R}^{1 \times h}$ es el vector de sesgo.
- ▶ El operador $\odot$ representa el producto de Hadamard (elemento a elemento).

Gated Recurrent Unit (GRU)

- ▶ Comparando con una RNN estándar:
 - ▶ Si $R_t \approx 1$, se recupera el comportamiento de una RNN clásica.
 - ▶ Si $R_t \approx 0$, se ignora el estado oculto previo H_{t-1} , y $\tilde{H}_t$ depende únicamente de la entrada X_t .
- ▶ De esta forma, la **reset gate** controla la influencia del pasado, permitiendo reiniciar parcialmente la memoria de la red.

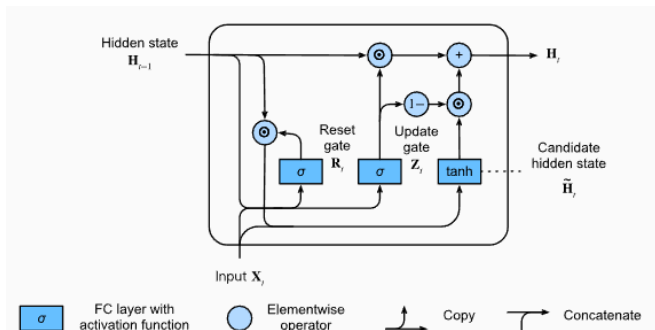


Gated Recurrent Unit (GRU)

Estado Oculto (Hidden State).

- ▶ Para definir el estado oculto final $H_t \in \mathbb{R}^{n \times h}$, se incorpora el efecto de la **update gate** Z_t .
- ▶ Esta compuerta controla cuánto del nuevo estado oculto proviene del estado anterior H_{t-1} y cuánto del estado candidato $\tilde{H}_t$.

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$



Interpretación del Estado Oculto en la GRU.

- ▶ Si $Z_t \approx 1$:
 - ▶ El modelo retiene el estado previo H_{t-1} .
 - ▶ La contribución del nuevo candidato $\tilde{H}_t$ se ignora.
 - ▶ Efectivamente, el paso de tiempo t se omite en la cadena de dependencias.
- ▶ Si $Z_t \approx 0$:
 - ▶ El nuevo estado H_t se aproxima al candidato $\tilde{H}_t$.
 - ▶ Se enfatiza la incorporación de la nueva información proveniente de X_t .
- ▶ De este modo, la **update gate** permite regular dinámicamente el equilibrio entre memoria y actualización.

Número de Parámetros en una GRU.

- ▶ Una GRU contiene tres conjuntos de parámetros principales:
 1. **Reset gate** (R_t).
 2. **Update gate** (Z_t).
 3. **Estado candidato** ($\tilde{H}_t$).
- ▶ Cada uno requiere:
 - ▶ Matriz de pesos para la entrada: $W_{x*} \in \mathbb{R}^{d \times h}$
 - ▶ Matriz de pesos para el estado oculto previo: $W_{h*} \in \mathbb{R}^{h \times h}$
 - ▶ Vector de sesgos: $b_* \in \mathbb{R}^h$
- ▶ Total por cada puerta:

$$d \cdot h + h \cdot h + h$$

Fórmula General del Número de Parámetros en la GRU.

- ▶ Como existen tres bloques de parámetros (R_t , Z_t , $\tilde{H}_t$):

$$N_{\text{GRU}} = 3 \times (d \cdot h + h \cdot h + h)$$

- ▶ d : número de características de entrada.
- ▶ h : número de unidades ocultas.

Interpretación

Este número crece cuadráticamente con h y linealmente con d , lo cual influye directamente en la capacidad de memoria y en el costo computacional de la red.

Ejemplo Numérico con Capa de Salida.

- ▶ Suponga:
 - ▶ $d = 10$ entradas,
 - ▶ $h = 8$ unidades ocultas,
 - ▶ $m = 4$ salidas.
- ▶ Parámetros internos de la GRU:

$$N_{\text{GRU}} = 3 \times (d \cdot h + h \cdot h + h)$$

$$N_{\text{GRU}} = 3 \times (10 \cdot 8 + 8 \cdot 8 + 8) = 3 \times 152 = 456$$

- ▶ Parámetros de la capa de salida:

$$N_{\text{salida}} = h \cdot m + m = 8 \cdot 4 + 4 = 36$$

- ▶ **Total:**

$$N_{\text{total}} = 456 + 36 = 492$$

Ventajas y Limitaciones de GRU.

Ventajas:

- ▶ Menor complejidad que LSTM (menos parámetros, más rápida de entrenar).
- ▶ Permite preservar información a largo plazo gracias a la puerta de actualización.
- ▶ Menor riesgo de gradientes desvanecientes que RNN Vanilla.
- ▶ Buen rendimiento en tareas de NLP, series temporales y secuencias largas.

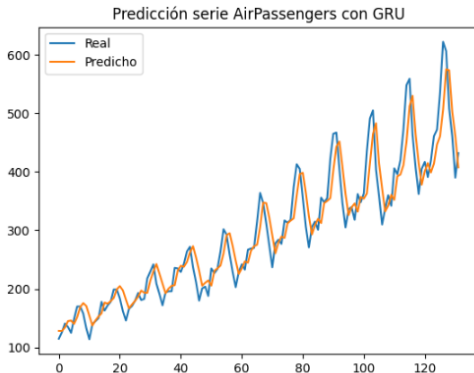
Limitaciones:

- ▶ Menos control fino sobre la información que LSTM, ya que no hay estado de celda separado.
- ▶ Puede ser insuficiente para secuencias extremadamente largas o complejas.
- ▶ No garantiza la eliminación total de gradientes desvanecientes/explosivos.

Gated Recurrent Unit (GRU)

Ejemplo Python

- ▶ Ver código: **Ejemplo GRU**.
- ▶ Evolución de la serie temporal AirPassengers con GRU.



Transfer Learning.

- ▶ El *transfer learning* consiste en reutilizar el conocimiento aprendido en una tarea **fuente** para mejorar el rendimiento en una tarea **destino**.
- ▶ Sea:

$$\mathcal{D}_S = \{(x_i^S, y_i^S)\}_{i=1}^{N_S}, \quad \mathcal{D}_T = \{(x_j^T, y_j^T)\}_{j=1}^{N_T}$$

donde $\mathcal{D}_S$ es el dataset fuente y $\mathcal{D}_T$ el dataset destino.

- ▶ Se entrena primero un modelo f_{θ_S} en $\mathcal{D}_S$ y luego se adapta a $\mathcal{D}_T$.

Formulación Matemática.

- ▶ En la **tarea fuente**, el modelo busca optimizar los parámetros minimizando la *pérdida esperada*:

$$\theta_S^* = \arg \min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}_S} [\ell(f_{\theta}(x), y)]$$

$\mathbb{E}$ denota el valor esperado bajo la distribución de datos $\mathcal{D}_S$.

- ▶ En la **tarea destino**, se aprovecha el conocimiento previo (parámetros θ_S^*) como punto de partida:

$$\theta_T^* = \arg \min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}_T} [\ell(f_{\theta}(x), y)], \quad \theta \approx \theta_S^*$$

donde $\mathbb{E}$ se calcula ahora bajo la distribución de datos $\mathcal{D}_T$.

Esta transferencia permite **reducir la necesidad de grandes cantidades de datos** en $\mathcal{D}_T$.

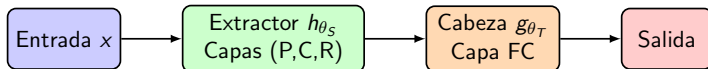
Transferencia de Representaciones.

- ▶ Un modelo profundo puede expresarse como:

$$f_{\theta}(x) = g_{\theta_T}(h_{\theta_S}(x))$$

donde:

- ▶ $h_{\theta_S}(x)$: **extractor de características** preentrenado (capas convolucionales o recurrentes).
- ▶ g_{θ_T} : **cabeza de la red** (generalmente capas *fully connected*) que actúa como clasificador o regresor en la tarea destino.



- ▶ Estrategias:
 1. **Feature Extraction:** congelar h_{θ_S} y entrenar únicamente g_{θ_T} .
 2. **Fine-Tuning:** ajustar parcialmente h_{θ_S} junto con g_{θ_T} .

Ejemplo Numérico.

- ▶ **Tarea fuente:** clasificación en *ImageNet*

$$N_S \approx 10^6 \text{ imágenes, } C_S = 1000 \text{ clases.}$$

- ▶ **Tarea destino:** clasificación de imágenes médicas

$$N_T = 2000 \text{ imágenes, } C_T = 5 \text{ clases.}$$

- ▶ **Transferencia de representaciones - Feature extraction:**

- ▶ Con ResNet-50 ([ver](#)), la última capa convolucional produce 2048 mapas de características de tamaño 7×7 .
- ▶ Luego, el global average pooling promedia cada mapa de 7×7 a un solo número, resultando en un vector de 2048 dimensiones.

$$h_{\theta_S} \in \mathbb{R}^{2048}$$

- ▶ La estrategia **Feature Extraction**: congela h_{θ_S} y entrenar únicamente g_{θ_T} :

$$g_{\theta_T}(z) = Wz + b$$

Con:

$$W \in \mathbb{R}^{C_T \times 2048} \text{ y } b \in \mathbb{R}^{C_T}$$

.

- ▶ **Número de parámetros entrenados en la tarea destino:**

$$N_{\text{salida}} = C_T \cdot 2048 + C_T = 5 \cdot 2048 + 5 = 10.245$$

- ▶ En vez de **millones de parámetros**, solo se ajustan $\approx 10^4$, reduciendo drásticamente la necesidad de datos y cómputo.

Conclusiones.

- ▶ El *transfer learning* permite reutilizar representaciones entrenadas en grandes datasets fuente para tareas con pocos datos.
- ▶ Formalmente, equivale a iniciar la optimización de θ_T desde θ_S^* .
- ▶ Estrategias comunes: **extracción de características** y **fine-tuning**.
- ▶ Ventajas:
 - ▶ Acelera el entrenamiento.
 - ▶ Reduce el sobreajuste.
 - ▶ Mejora el rendimiento en datasets pequeños.

Fine-Tuning.

- ▶ El fine-tuning consiste en adaptar un modelo previamente entrenado (“pretrained”) a una tarea objetivo, **iniciando la optimización desde los pesos preentrenados θ^*** .
- ▶ Se aprovecha la representación aprendida en la etapa previa para acelerar entrenamiento y mejorar la generalización en la tarea objetivo.

Notación.

Sea:

- ▶ $\mathcal{D}_{\text{pre}}$: dataset de preentrenamiento (grande, genérico).
- ▶ $\mathcal{D}_{\text{tgt}}$: dataset de la tarea objetivo (**target**, pequeño, específico).
- ▶ Modelo paramétrico: $f(x; \theta)$, con parámetros $\theta \in \mathbb{R}^p$.
- ▶ Función de pérdida general: $\ell(y, \hat{y})$.
- ▶ Pérdida empírica en la tarea objetivo:

$$L_{\text{tgt}}(\theta) = \frac{1}{N} \sum_{(x,y) \in \mathcal{D}_{\text{tgt}}} \ell(y, f(x; \theta)).$$

A partir del modelo preentrenado en una tarea fuente:

$$\theta^* = \arg \min_{\theta} L_{\text{pre}}(\theta).$$

En fine-tuning

- ▶ Primero se inicializan los parámetros como:

$$\theta^{(0)} = \theta^*,$$

- ▶ y luego se optimiza sobre el dataset destino:

$$\theta_{\text{tgt}}^* = \arg \min_{\theta} L_{\text{tgt}}(\theta).$$

Esto permite:

- ▶ adaptar el conocimiento aprendido en $\mathcal{D}_{\text{pre}}$ para que el modelo se especialice en $\mathcal{D}_{\text{tgt}}$.
- ▶ reducir la necesidad de datos y cómputo, y mejora la generalización en la tarea objetivo.

Estrategias de Fine-Tuning:

- ▶ Tras el preentrenamiento se tienen parámetros θ^* en un modelo $f(x; \theta)$.
- ▶ El fine-tuning decide **qué parámetros actualizar** al optimizar sobre $\mathcal{D}_{\text{tgt}}$.
- ▶ Diferentes estrategias buscan equilibrar:
 - ▶ Precisión en la tarea objetivo.
 - ▶ Costo computacional.
 - ▶ Cantidad de datos disponibles.
- ▶ Existen **enfoques clásicos**: Linear probe, Head-only, Partial, Full y
- ▶ **enfoques modernos**: Adapters, LoRA, Prompt tuning.

Solo cabeza (Linear Probe).

- ▶ Se congela el **backbone** $h_{\theta^*}(x)$ (extractor de características).
- ▶ Se entrena **únicamente una capa lineal de salida**:

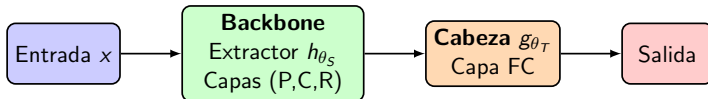
$$f(x; \theta) = W h_{\theta^*}(x) + b,$$

con parámetros $\theta = \{W, b\}$.

- ▶ Optimización:

$$W^*, b^* = \arg \min_{W, b} \frac{1}{N} \sum_{(x, y) \in \mathcal{D}_{\text{tgt}}} \ell(y, W h_{\theta^*}(x) + b).$$

- ▶ **Ventaja**: muy eficiente, requiere pocos datos.
- ▶ **Limitación**: poca capacidad de adaptación.



Ejemplo: Linear Probe (solo cabeza).

- ▶ Se congela el backbone y se entrena sólo la capa de salida.
- ▶ Parámetros entrenables:

$$W \in \mathbb{R}^{m \times d_\phi}, \quad \# = md_\phi + m$$

- ▶ Para $d_\phi = 768$, $m = 10$:

$$\# = 10 \cdot 768 + 10 = 7\,690$$

Ventaja: mínimo costo de entrenamiento.

Limitación: poca capacidad de adaptación.

Cabeza nueva (Head-Only).

- ▶ Se reemplazan y entrenan las **últimas capas** de la red.
- ▶ El backbone $h_{\theta^*}(x)$ se mantiene congelado, pero **se entrena un clasificador más complejo** g_{ϕ} , ejemplo un MLP.
- ▶ Modelo:

$$f(x; \theta) = g_{\phi}(h_{\theta^*}(x)).$$

- ▶ Optimización sobre ϕ únicamente:

$$\phi^* = \arg \min_{\phi} L_{\text{tgt}}(\phi \mid \theta^*).$$

- ▶ **Ventaja:** más flexible que linear probe.
- ▶ **Limitación:** la base no se adapta si el dominio difiere mucho.

Ejemplo: Head-Only (cabeza nueva).

- ▶ Se reemplazan las últimas capas (ej. clasificación + MLP final) y se entrenan desde cero.
- ▶ Supóngase que la última capa oculta es $d_\phi = 768$ y se añade un MLP intermedio de 512 neuronas.
- ▶ Parámetros entrenables:

$$W_1 \in \mathbb{R}^{512 \times 768}, \quad W_2 \in \mathbb{R}^{10 \times 512}.$$

- ▶ Conteo:

$$\# = (512 \cdot 768 + 512) + (10 \cdot 512 + 10) = 393\,226.$$

Ventaja: mayor capacidad que linear probe.

Limitación: más parámetros que entrenar.

Últimas k capas + cabeza (Partial Fine-Tune).

- ▶ Se descongelan las últimas k capas del backbone, además de la cabeza.
- ▶ El modelo se puede escribir como:

$$f(x; \theta) = g_{\phi}(h_{\theta_{[L-k:L]}}(h_{\theta_{[1:L-k]}}(x))),$$

donde solo se optimizan $\theta_{[L-k:L]}$ y ϕ .

- ▶ Optimización:

$$\theta_{[L-k:L]}^*, \phi^* = \arg \min L_{\text{tgt}}.$$

- ▶ **Ventaja:** balance entre costo y capacidad de adaptación.
- ▶ **Limitación:** requiere más datos que head-only.

Ejemplo: Partial Fine-Tuning.

- ▶ Se descongelan las últimas k capas del backbone junto con la cabeza.
- ▶ Supóngase $k = 2$ capas con pesos 768×768 .
- ▶ Parámetros entrenables en las dos capas:

$$2 \cdot 768^2 \approx 1.18 \times 10^6.$$

- ▶ Más la cabeza lineal:

$$7690.$$

- ▶ Total:

$$\approx 1.19 \times 10^6.$$

Ventaja: balance entre costo y capacidad.

Ajuste completo (Full Fine-Tune).

- ▶ Todos los parámetros se inicializan en $\theta^{(0)} = \theta^*$.
- ▶ Se optimizan todos los pesos:

$$\theta_{\text{tgt}}^* = \arg \min_{\theta} L_{\text{tgt}}(\theta).$$

- ▶ **Ventaja:** máxima capacidad de adaptación al dominio destino.
- ▶ **Limitación:** alto costo computacional y riesgo de sobreajuste si $|\mathcal{D}_{\text{tgt}}|$ es pequeño.
- ▶ Usado cuando el dataset destino es grande o muy diferente del dataset fuente.

Ejemplo: Full Fine-Tuning.

- ▶ Se entrenan todos los parámetros del modelo.
- ▶ Supóngase un backbone de $L = 12$ capas con $W^{(l)} \in \mathbb{R}^{768 \times 768}$.
- ▶ Parámetros por capa:

$$768^2 \approx 589\,824.$$

- ▶ Total para 12 capas:

$$12 \cdot 589\,824 \approx 7.1 \times 10^6.$$

- ▶ Más la cabeza lineal: 7 690 (despreciable en comparación).

Ventaja: máxima flexibilidad.

Limitación: muy costoso, riesgo de sobreajuste.

Adaptación mediante capas ligeras (Adapters).

- ▶ Idea: **añadir módulos pequeños (“adapters”) en paralelo a las capas originales congeladas** para especializar el modelo en la tarea objetivo.
- ▶ Fórmula típica:

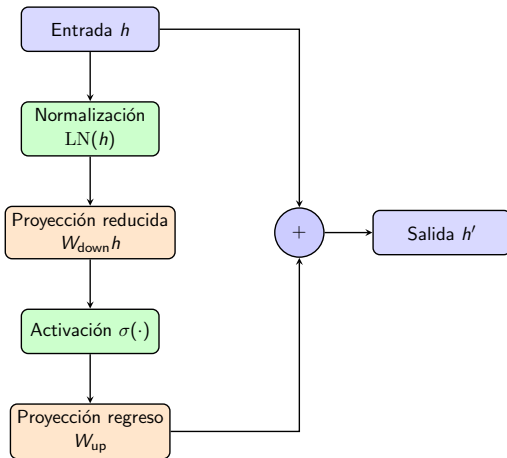
$$h' = h + W_{\text{up}} \sigma(W_{\text{down}} \text{LN}(h)),$$

donde:

- ▶ $h \in \mathbb{R}^d$: representación de entrada de dimensión d .
- ▶ $\text{LN}(h)$: normalización de capa (LayerNorm).
- ▶ $W_{\text{down}} \in \mathbb{R}^{r \times d}$: proyección a un espacio reducido (“bottleneck”).
- ▶ $W_{\text{up}} \in \mathbb{R}^{d \times r}$: proyección de regreso a la dimensión original.
- ▶ $r \ll d$: el tamaño del cuello de botella controla cuántos parámetros se entrenan.
- ▶ σ representa una función de activación no lineal.

- Solo se entrenan W_{down} , W_{up} ; el resto del modelo permanece congelado.

$$h' = h + W_{\text{up}} \sigma(W_{\text{down}} \text{LN}(h))$$



- ▶ **Ventaja:** se reduce drásticamente el número de parámetros entrenables ($O(dr)$ frente a $O(d^2)$).
- ▶ **Aplicación:** ampliamente usados en NLP (BERT, GPT), visión (ViT) y modelos multimodales.

Ejemplo: Adapters.

- ▶ Se agregan capas de proyección reducida $r \ll d_\phi$.
- ▶ Fórmula:

$$h' = h + W_{\text{up}} \sigma(W_{\text{down}} h).$$

- ▶ Supóngase $r = 64$, $d_\phi = 768$:

$$W_{\text{down}} \in \mathbb{R}^{64 \times 768}, \quad W_{\text{up}} \in \mathbb{R}^{768 \times 64}.$$

- ▶ Parámetros:

$$64 \cdot 768 + 768 \cdot 64 = 98\,304$$

por capa adaptada.

Ventaja: eficiente, se añaden pocos parámetros.

Adaptación de bajo rango (Low-Rank Adaptation – LoRA).

- ▶ En modelos preentrenados muy grandes, **las matrices de pesos W de ciertas capas** son demasiado costosas de ajustar completamente.
- ▶ Estrategia: mantener W fijo y añadir una actualización de bajo rango ΔW .

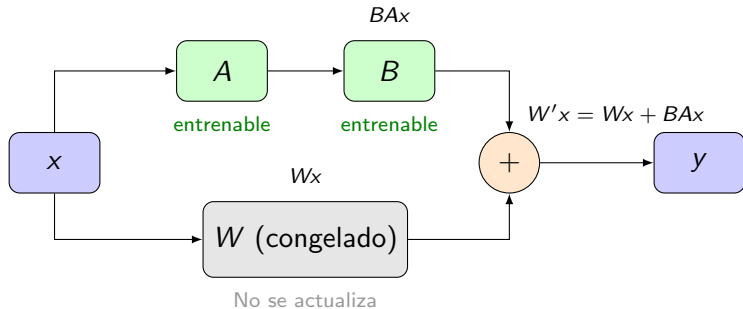
$$W' = W + \Delta W, \quad \Delta W = BA,$$

con $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ y $r \ll \min(d, k)$.

- ▶ Durante el entrenamiento, solo se optimizan A y B , mientras que W (los parámetros originales de la capa) permanecen congelados.

Fine-Tuning

- ▶ Solo se optimizan A y B , mientras
- ▶ W , los parámetros originales de la capa) permanecen congelados.



- ▶ **Ventaja:** el número de parámetros entrenables se reduce en órdenes de magnitud.
- ▶ **Aplicación:** ajuste eficiente de LLMs, modelos de visión y multimodales.

Ejemplo: LoRA (Low-Rank Adaptation).

- ▶ Se congela $W \in \mathbb{R}^{768 \times 768}$ y se entrena $\Delta W = BA$.

- ▶ Con $r = 4$:

$$B \in \mathbb{R}^{768 \times 4}, \quad A \in \mathbb{R}^{4 \times 768}.$$

- ▶ Parámetros:

$$2 \cdot r \cdot 768 = 6\,144$$

por capa adaptada.

- ▶ Mucho menor que reentrenar 589 824 de la matriz completa.

Ventaja: ajuste eficiente en LLMs y CNNs grandes.

Adaptación mediante entradas aprendidas (Prompt / Prefix Tuning).

- ▶ Se congelan los parámetros del modelo θ^* .
- ▶ Se aprenden vectores adicionales (*prompts* p_i) concatenados a la entrada:

$$\tilde{x} = [p_1, p_2, \dots, p_m, x],$$

donde p_i son vectores (embeddings) entrenables.

- ▶ Durante el entrenamiento: no se actualizan los pesos del modelo, **solo se ajustan los embeddings del prompt**.
- ▶ Optimización:

$$P^* = \arg \min_P L_{\text{tgt}}(\tilde{x}; \theta^*).$$

- ▶ **Ventaja:** extremadamente ligero (solo unos pocos miles de parámetros).
- ▶ **Aplicación:** NLP, visión multimodal, generación de texto.

Formalmente

- ▶ Si el modelo recibe una entrada:

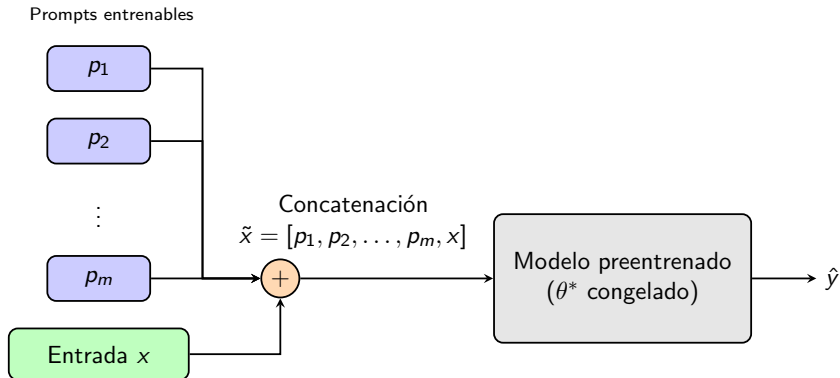
$$x = [p_1, p_2, \dots, p_k, x_1, x_2, \dots, x_n]$$

- ▶ p_i : **tokens del prompt** (aprendidos)
 - ▶ x_i : **datos reales**
- ▶ El entrenamiento optimiza:

$$\min_p \mathcal{L}(f([p, x]), y)$$

- ▶ Solo se ajusta p
- ▶ **Características clave**
 - ▶ Muy pocos parámetros entrenables
 - ▶ No modifica el modelo base
 - ▶ Bajo costo computacional
 - ▶ Fácil de almacenar (solo el prompt)

(Prompts p_i) concatenados.



Ejemplo: Prompt / Prefix Tuning.

- ▶ Se añaden m_p vectores aprendidos a la entrada.
- ▶ Supóngase $m_p = 20$, cada prompt de dimensión $d_\phi = 768$.
- ▶ Parámetros entrenables:

$$20 \cdot 768 = 15\,360.$$

- ▶ Backbone permanece congelado.

Ventaja: extremadamente ligero, ideal para LLMs y NLP.

Resumen Comparativo.

Estrategia	Capas entrenadas	Parámetros	Uso típico
Linear probe	Solo salida	Muy pocos	Dataset destino muy pequeño
Head-only	Últimas capas	Pocos	Dataset moderado
Partial FT	Últimas k capas	Medio	Balance costo/adaptación
Full FT	Todas	Muchos	Dataset grande o distinto
Adapters	Capas extra pequeñas	Muy pocos	Modelos grandes
LoRA	Factores de bajo rango	Muy pocos	LLMs / visión
Prompt tuning	Vectores de entrada	Poquísimos	NLP / multimodal

Data Augmentation.

Es una técnica que consiste en generar de manera artificial **nuevas muestras de entrenamiento a partir de las ya existentes**, aplicando transformaciones como rotaciones, traslaciones, recortes, cambios de color o ruido.

Formalmente:

- ▶ para un ejemplo (x, y) se aplica una transformación T tomada de una familia de transformaciones $\mathcal{T}$:

$$(x', y') = (T(x), y), \quad T \sim \mathcal{T}.$$

- ▶ **Objetivo:** aumentar la diversidad del conjunto de datos sin necesidad de recolectar datos adicionales.
- ▶ **Uso:** mejorar la generalización del modelo, reducir el sobreajuste (overfitting) y aumentar el rendimiento en tareas como visión por computadora, procesamiento de lenguaje natural y audio.

Formulación Probabilística.

- ▶ La distribución original de los datos es:

$$p_{\text{data}}(x, y).$$

- ▶ Al aplicar *data augmentation*, se genera una nueva distribución:

$$p_{\text{aug}}(x, y) = \int_{\mathcal{T}} p(T) p_{\text{data}}(T^{-1}(x), y) dT,$$

donde:

- ▶ $\mathcal{T}$ = conjunto de transformaciones posibles.
- ▶ $p(T)$ = probabilidad de aplicar cada transformación.
- ▶ $T^{-1}(x)$ = ejemplo original que al transformarse da x .
- ▶ **Interpretación:** el modelo entrena sobre una distribución **enriquecida**, con múltiples vistas de x , manteniendo fijo y .

Ejemplo Matemático en Imágenes.

- ▶ Sea una imagen $x \in \mathbb{R}^{H \times W \times C}$, con H = altura, W = ancho y C = canales.
- ▶ **Rotación:** Para un ángulo θ :

$$x'(u, v) = x \left(R_{\theta}^{-1} \cdot \begin{bmatrix} u \\ v \end{bmatrix} \right), \text{ donde: } R_{\theta} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

Aquí (u, v) son coordenadas en la imagen transformada.

- ▶ **Cambio de Brillo:**

$$x' = \alpha x, \quad \alpha \sim \mathcal{U}(a, b).$$

donde α es un factor aleatorio que oscurece ($\alpha < 1$) o aclara ($\alpha > 1$) la imagen.

Idea: Cada transformación T genera una nueva vista $x' = T(x)$, manteniendo fija la etiqueta y .

Ejemplo Numérico.

- ▶ Dataset original: $N = 1000$ imágenes.
- ▶ Se definen 5 transformaciones comunes:
 - ▶ Rotación
 - ▶ Traslación
 - ▶ Escalado
 - ▶ Inversión horizontal
 - ▶ Jitter de color
- ▶ Cada imagen produce en promedio $k = 10$ variaciones sintéticas (2 por transformación).
- ▶ Tamaño efectivo del dataset:

$$N_{\text{aug}} = N \times k = 1000 \times 10 = 10000$$

- ▶ **Beneficio:** El modelo entrena sobre un conjunto $\sim 10\times$ más grande, con mayor diversidad, reduciendo el sobreajuste y mejorando la generalización.

Síntesis

- ▶ El *data augmentation* puede entenderse como la aplicación de transformaciones **estocásticas y controladas** sobre los datos de entrada:

$$(x', y) = (T(x), y), \quad T \sim \mathcal{T}.$$

- ▶ **Objetivo principal:** enriquecer la distribución de entrenamiento, reduciendo el sobreajuste y favoreciendo la capacidad de generalización.
- ▶ **Relevancia práctica:** es un componente esencial en el **deep learning moderno**, con impacto destacado en:
 - ▶ Visión por computadora (rotaciones, cambios de color).
 - ▶ Procesamiento de audio (ruido, pitch shifting).
 - ▶ Procesamiento de texto (paráfrasis, inserción de sinónimos).

Conclusión: El *data augmentation* convierte datasets limitados en conjuntos más ricos y representativos, siendo clave para el éxito de modelos robustos.

Gracias por su atención.

